Geth Database 명령어 상세 분석

`geth db` 명령어는 Ethereum 노드의 저수준 데이터베이스 작업을 수행하는 강력한 도구입니다.

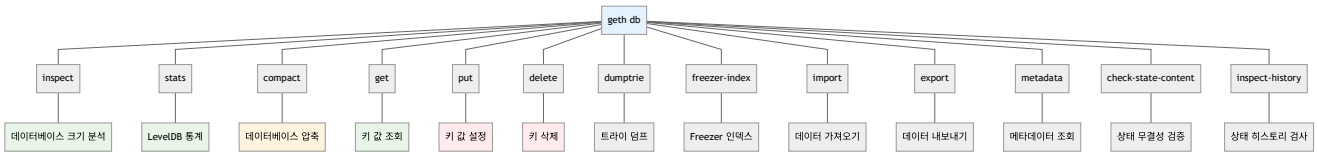
> **명령어:** `geth db <subcommand>`

> **진입점:** `dbCommand` → 각 서브명령어 함수

> **소스 파일:** `cmd/geth/dbcmd.go`

> **카테고리:** 데이터베이스 관리

🚀 전체 명령어 구조



📋 명령어 정의 및 구조

메인 DB 명령어

```
// cmd/geth/dbcmd.go

dbCommand = &cli.Command{
```

```
Name: "db",
Usage: "Low level database operations",
ArgsUsage: "",
Subcommands: []*cli.Command{
dbInspectCmd,
dbStatCmd,
dbCompactCmd,
dbGetCmd,
dbDeleteCmd,
dbPutCmd,
dbGetSlotsCmd,
dbDumpFreezerIndex,
dbImportCmd,
dbExportCmd,
dbMetadataCmd,
dbCheckStateContentCmd,
dbInspectHistoryCmd,
},
}
```

📋 서브명령어별 상세 분석

1. `geth db inspect` - 데이터베이스 크기 분석

```
dbInspectCmd = &cli.Command{
Action: inspect,
Name: "inspect",
ArgsUsage: "<prefix> <start>",
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Usage: "Inspect the storage size for each type of data in the database",
Description: `This commands iterates the entire database. If the optional 'prefi
}


// inspect 함수
func inspect(ctx *cli.Context) error {
var (
prefix []byte
start []byte
)
// 1. 인수 파싱
if ctx.NArg() > 2 {
return fmt.Errorf("Max 2 arguments: %v", ctx.Command.ArgsUsage)
}
if ctx.NArg() >= 1 {
if d, err := hexutil.Decode(ctx.Args().Get(0)); err != nil {
return fmt.Errorf("failed to hex-decode 'prefix': %v", err)
```

```
} else {
prefix = d
}
}
if ctx.NArg() >= 2 {
if d, err := hexutil.Decode(ctx.Args().Get(1)); err != nil {
return fmt.Errorf("failed to hex-decode 'start': %v", err)
} else {
start = d
}
}
// 2. 노드 스택 및 데이터베이스 생성
stack, _ := makeConfigNode(ctx)
defer stack.Close()
db, err := utils.MakeChainDatabase(ctx, stack, true) // readonly
if err != nil {
return err
}
defer db.Close()
// 3. 데이터베이스 검사 실행
return rawdb.InspectDatabase(db, prefix, start)
}
```

사용 예시:

```
# 전체 데이터베이스 분석
$ geth db inspect
INFO [날짜] Inspecting database
Database version: 8
Key-Value store: LevelDB
Database size: 750.5 GB

=== Database Analysis ===
Headers: 15.2 GB (2.0%)
Bodies: 125.8 GB (16.8%)
Receipts: 245.6 GB (32.7%)
State: 298.4 GB (39.8%)
Codes: 45.2 GB (6.0%)
Tries: 20.3 GB (2.7%)

# 특정 접두사로 제한
$ geth db inspect 0x68 0x6800
Inspecting keys with prefix 0x68 starting from 0x6800
Found 12,345 keys
Total size: 2.3 GB
```

2. geth db stats - LevelDB 통계

```
dbStatCmd = &cli.Command{
Action: dbStats,
Name: "stats",
Usage: "Print leveldb statistics",
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
}

// dbStats 함수
func dbStats(ctx *cli.Context) error {
// 1. 데이터베이스 열기
stack, _ := makeConfigNode(ctx)
defer stack.Close()
db, err := utils.MakeChainDatabase(ctx, stack, true)
if err != nil {
return err
}
defer db.Close()
// 2. LevelDB 통계 가져오기
if ldb, ok := db.(*ethdb.LDBDatabase); ok {
stats, err := ldb.LDB().GetProperty("leveldb.stats")
if err != nil {
```

```
return err
}
fmt.Println(stats)
// 3. 추가 통계 정보
sstables, err := ldb.LDB().GetProperty("leveldb.sstables")
if err != nil {
return err
}
fmt.Println("\\n=== SSTable Information ===")
fmt.Println(sstables)
}
return nil
}
```

사용 예시:

```
$ geth db stats
Compactions
Level | Tables | Size(MB) | Time(sec) | Read(MB) | Write(MB)
-------+------------+---------------+---------------+---------------+-----------
0 | 0 | 0.00000 | 0.00000 | 0.00000 | 0.00000
1 | 8 | 1024.00000 | 45.12300 | 2048.00000 | 1024.00000
2 | 64 | 8192.00000 | 234.56700 | 16384.00000 | 8192.00000
3 | 512 | 65536.00000 | 1234.56700 | 131072.00000 | 65536.00000
```

```
=== SSTable Information ===
--- level 0 ---
--- level 1 ---
12345678: 1024000['abc123'..'def456']
23456789: 2048000['def456'..'ghi789']
--- level 2 ---
...
```

3. geth db compact - 데이터베이스 압축

```
dbCompactCmd = &cli.Command{
Action: dbCompact,
Name: "compact",
Usage: "Compact leveldb database. WARNING: May take a very long time",
Flags: slices.Concat([]cli.Flag{
utils.CacheFlag,
utils.CacheDatabaseFlag,
}, utils.NetworkFlags, utils.DatabaseFlags),
Description: `This command performs a database compaction.
WARNING: This operation may take a very long time to finish, and may cause datab
corruption if it is aborted during execution'!`,
}
```

```go
// dbCompact 함수
func dbCompact(ctx *cli.Context) error {
// 1. 데이터베이스 열기 (쓰기 모드)
stack, _ := makeConfigNode(ctx)
defer stack.Close()
db, err := utils.MakeChainDatabase(ctx, stack, false) // readonly = false
if err != nil {
return err
}
defer db.Close()
// 2. 압축 시작 전 통계
fmt.Println("Starting database compaction...")
start := time.Now()
// 3. 중단 신호 처리
interrupt := make(chan os.Signal, 1)
signal.Notify(interrupt, syscall.SIGINT, syscall.SIGTERM)
// 4. 압축 실행
go func() {
<-interrupt
fmt.Println("\\nCompaction interrupted! Database may be corrupted!")
os.Exit(1)
}()
```

```
if err := db.Compact(nil, nil); err != nil {
return fmt.Errorf("compaction failed: %v", err)
}
// 5. 완료 통계
fmt.Printf("Database compaction completed in %v\\n", time.Since(start))
return nil
}
```

사용 예시:

```
$ geth db compact
Starting database compaction...
WARNING: This operation may take several hours to complete.
Press Ctrl+C to abort (may cause corruption).

Compacting level 0...
Compacting level 1...
Compacting level 2...
...
Database compaction completed in 2h45m30s
```

4. geth db get - 키 값 조회

```
dbGetCmd = &cli.Command{
```

```
Action: dbGet,
Name: "get",
Usage: "Show the value of a database key",
ArgsUsage: "<hex-encoded key>",
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Description: "This command looks up the specified database key from the database
}


// dbGet 함수
func dbGet(ctx *cli.Context) error {
// 1. 인수 검증
if ctx.NArg() != 1 {
return fmt.Errorf("required arguments: %v", ctx.Command.ArgsUsage)
}
// 2. 키 디코딩
key, err := hexutil.Decode(ctx.Args().Get(0))
if err != nil {
return fmt.Errorf("could not decode the key: %v", err)
}
// 3. 데이터베이스 열기
stack, _ := makeConfigNode(ctx)
defer stack.Close()
db, err := utils.MakeChainDatabase(ctx, stack, true)
```

```
if err != nil {
return err
}
defer db.Close()
// 4. 값 조회
data, err := db.Get(key)
if err != nil {
return err
}
// 5. 결과 출력
fmt.Printf("Key: %s\\n", hexutil.Encode(key))
fmt.Printf("Value: %s\\n", hexutil.Encode(data))
fmt.Printf("Size: %d bytes\\n", len(data))
// 6. 가능한 경우 디코딩 시도
if len(data) > 0 {
fmt.Println("\\n=== Decoding Attempts ===")
// RLP 디코딩 시도
if rlpData, err := rlp.DecodeBytes(data, new(interface{})); err == nil {
fmt.Printf("RLP decoded: %+v\\n", rlpData)
}
// 해시 확인
if len(data) == 32 {
fmt.Printf("As hash: %s\\n", common.BytesToHash(data).Hex())
```

```
}
// 주소 확인
if len(data) == 20 {
fmt.Printf("As address: %s\\n", common.BytesToAddress(data).Hex())
}
}
return nil
}
```

사용 예시:

```
$ geth db get 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890ab
Key: 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef
Value: 0xf86c808504a817c800825208941234567890abcdef1234567890abcdef12345678880d
Size: 108 bytes

=== Decoding Attempts ===
RLP decoded: [0 1250000000 21000 0x1234567890abcdef1234567890abcdef12345678
```

5. `geth db put` - 키 값 설정 (위험)

```
dbPutCmd = &cli.Command{
Action: dbPut,
```

```
Name: "put",

Usage: "Set the value of a database key (WARNING: may corrupt your database)",

ArgsUsage: "<hex-encoded key> <hex-encoded value>",

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: `This command sets a given database key to the given value.

WARNING: This is a low-level operation which may cause database corruption!`,

}



// dbPut 함수

func dbPut(ctx *cli.Context) error {

// 1. 인수 검증

if ctx.NArg() != 2 {

return fmt.Errorf("required arguments: %v", ctx.Command.ArgsUsage)

}

// 2. 키와 값 디코딩

key, err := hexutil.Decode(ctx.Args().Get(0))

if err != nil {

return fmt.Errorf("could not decode the key: %v", err)

}

value, err := hexutil.Decode(ctx.Args().Get(1))

if err != nil {

return fmt.Errorf("could not decode the value: %v", err)

}
```

```
// 3. 경고 메시지 및 확인

fmt.Println("⚠️ WARNING: This operation may corrupt your database!")

fmt.Printf("Key: %s\\n", hexutil.Encode(key))

fmt.Printf("Value: %s\\n", hexutil.Encode(value))

fmt.Print("Are you sure you want to continue? (yes/no): ")

input, err := prompt.Stdin.PromptInput("")

if err != nil {

return err

}

if strings.ToLower(strings.TrimSpace(input)) != "yes" {

fmt.Println("Operation cancelled.")

return nil

}

// 4. 데이터베이스 열기 (쓰기 모드)

stack, _ := makeConfigNode(ctx)

defer stack.Close()

db, err := utils.MakeChainDatabase(ctx, stack, false)

if err != nil {

return err

}

defer db.Close()

// 5. 값 설정

if err := db.Put(key, value); err != nil {
```

```
return err
}
fmt.Println("✅ Key-value pair successfully written to database.")
return nil
}
```

6. geth db delete - 키 삭제 (위험)

```
dbDeleteCmd = &cli.Command{
Action: dbDelete,
Name: "delete",
Usage: "Delete a database key (WARNING: may corrupt your database)",
ArgsUsage: "<hex-encoded key>",
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Description: `This command deletes the specified database key from the database.
WARNING: This is a low-level operation which may cause database corruption!`,
}

// dbDelete 함수 (유사한 구조로 구현)
func dbDelete(ctx *cli.Context) error {
// 경고, 확인, 삭제 로직...
return db.Delete(key)
```

```
}
```

7. geth db dumptrie - 트라이 덤프

```
dbGetSlotsCmd = &cli.Command{
Action: dbDumpTrie,
Name: "dumptrie",
Usage: "Show the storage key/values of a given storage trie",
ArgsUsage: "<hex-encoded state root> <hex-encoded account hash> <hex-encoded sto
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Description: "This command looks up the specified database key from the database
}

// dbDumpTrie 함수
func dbDumpTrie(ctx *cli.Context) error {
// 1. 인수 파싱
if ctx.NArg() < 3 {
return fmt.Errorf("required arguments: %v", ctx.Command.ArgsUsage)
}
stateRoot, err := hexutil.Decode(ctx.Args().Get(0))
if err != nil {
return fmt.Errorf("invalid state root: %v", err)
}
```

```
accountHash, err := hexutil.Decode(ctx.Args().Get(1))
if err != nil {
return fmt.Errorf("invalid account hash: %v", err)
}
storageRoot, err := hexutil.Decode(ctx.Args().Get(2))
if err != nil {
return fmt.Errorf("invalid storage root: %v", err)
}
// 2. 선택적 인수
var start []byte
max := 1024 // 기본값
if ctx.NArg() > 3 {
start, err = hexutil.Decode(ctx.Args().Get(3))
if err != nil {
return fmt.Errorf("invalid start key: %v", err)
}
}
if ctx.NArg() > 4 {
max, err = strconv.Atoi(ctx.Args().Get(4))
if err != nil {
return fmt.Errorf("invalid max elements: %v", err)
}
}
```

```
// 3. 데이터베이스 및 트라이 설정
stack, _ := makeConfigNode(ctx)
defer stack.Close()
db, err := utils.MakeChainDatabase(ctx, stack, true)
if err != nil {
return err
}
defer db.Close()
triedb := trie.NewDatabase(db)
// 4. 스토리지 트라이 열기
storageTrie, err := trie.New(common.BytesToHash(storageRoot), triedb)
if err != nil {
return fmt.Errorf("could not open storage trie: %v", err)
}
// 5. 트라이 순회 및 출력
fmt.Printf("Storage trie dump for account %s\\n", hexutil.Encode(accountHash))
fmt.Printf("Storage root: %s\\n", hexutil.Encode(storageRoot))
fmt.Printf("Starting from: %s\\n", hexutil.Encode(start))
fmt.Printf("Max elements: %d\\n\\n", max)
it := trie.NewIterator(storageTrie.NodeIterator(start))
count := 0
for it.Next() && count < max {
fmt.Printf("Key: %s\\n", hexutil.Encode(it.Key))
```

```
fmt.Printf("Value: %s\\n", hexutil.Encode(it.Value))

fmt.Println("---")

count++

}

fmt.Printf("\\nDumped %d storage slots\\n", count)

return  nil

}
```

8. geth db metadata - 메타데이터 조회

```
dbMetadataCmd = &cli.Command{

Action: showMetaData,

Name: "metadata",

Usage: "Shows metadata about the chain status.",

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: "Shows metadata about the chain status.",

}


// showMetaData 함수

func  showMetaData(ctx *cli.Context) error {

// 1. 데이터베이스 열기

stack, _ := makeConfigNode(ctx)
```

```
defer stack.Close()

db, err := utils.MakeChainDatabase(ctx, stack, true)

if err != nil {

return err

}

defer db.Close()

// 2. 메타데이터 수집

fmt.Println("=== Chain Metadata ===")

// 헤드 블록 정보

headHash := rawdb.ReadHeadBlockHash(db)

if headHash != (common.Hash{}) {

headNumber := rawdb.ReadHeaderNumber(db, headHash)

if headNumber != nil {

fmt.Printf("Head block: #%d (%s)\\n", *headNumber, headHash.Hex())

}

}

// 제네시스 블록 정보

genesisHash := rawdb.ReadCanonicalHash(db, 0)

if genesisHash != (common.Hash{}) {

fmt.Printf("Genesis block: %s\\n", genesisHash.Hex())

}

// 체인 설정

if config := rawdb.ReadChainConfig(db, genesisHash); config != nil {
```

```
fmt.Printf("Chain ID: %d\\n", config.ChainID.Uint64())

fmt.Printf("Network ID: %d\\n", config.ChainID.Uint64())

}

// 동기화 상태

if  pivot := rawdb.ReadLastPivotNumber(db);  pivot != nil {

fmt.Printf("Fast sync pivot: #%d\\n", *pivot)

}

// 스냅샷 정보

if  root := rawdb.ReadSnapshotRoot(db);  root != (common.Hash{}) {

fmt.Printf("Snapshot root: %s\\n", root.Hex())

}

return  nil

}
```

🔧 RemoveDB 명령어

```
removedbCommand = &cli.Command{

Action: removeDB,

Name: "removedb",

Usage: "Remove blockchain and state databases",

ArgsUsage: "",

Flags: slices.Concat(utils.DatabaseFlags,

[]cli.Flag{removeStateDataFlag, removeChainDataFlag}),
```

```
Description: `Remove blockchain and state databases`,
}


// removeDB 함수
func removeDB(ctx *cli.Context) error {
// 1. 플래그 확인
removeState := ctx.Bool(removeStateDataFlag.Name)
removeChain := ctx.Bool(removeChainDataFlag.Name)
if !removeState && !removeChain {
return fmt.Errorf("either --remove.state or --remove.chain must be specified")
}
// 2. 경고 및 확인
fmt.Println("⚠️ WARNING: This will permanently delete database files!")
if removeState {
fmt.Println("- State database will be removed")
}
if removeChain {
fmt.Println("- Chain database will be removed")
}
fmt.Print("Are you sure? (yes/no): ")
input, err := prompt.Stdin.PromptInput("")
if err != nil {
return err
```

```
}
if strings.ToLower(strings.TrimSpace(input)) != "yes" {
fmt.Println("Operation cancelled.")
return nil
}
// 3. 노드 설정 로드
stack, _ := makeConfigNode(ctx)
defer stack.Close()
// 4. 데이터베이스 경로 확인
chainDataDir := stack.ResolvePath("chaindata")
stateDataDir := stack.ResolvePath("lightchaindata")
// 5. 삭제 실행
if removeChain {
fmt.Printf("Removing chain database: %s\\n", chainDataDir)
if err := os.RemoveAll(chainDataDir); err != nil {
return fmt.Errorf("failed to remove chain database: %v", err)
}
}
if removeState {
fmt.Printf("Removing state database: %s\\n", stateDataDir)
if err := os.RemoveAll(stateDataDir); err != nil {
return fmt.Errorf("failed to remove state database: %v", err)
}
```

```
}
fmt.Println("✅ Database removal completed.")
return nil
}
```

📊 실행 예시

1. 데이터베이스 크기 분석

```
$ geth db inspect
INFO [날짜] Inspecting database
Database version: 8
Key-Value store: LevelDB
Database size: 750.5 GB

=== Database Analysis ===
Headers: 15.2 GB (2.0%)
Bodies: 125.8 GB (16.8%)
Receipts: 245.6 GB (32.7%)
State: 298.4 GB (39.8%)
Codes: 45.2 GB (6.0%)
Tries: 20.3 GB (2.7%)
```

2. 특정 키 조회

```
$ geth db get 0x290decd9548b62a8d60345a988386fc84ba6bc95484008f6362f93160ef3

Key: 0x290decd9548b62a8d60345a988386fc84ba6bc95484008f6362f93160ef3e563

Value: 0xf90211a0123456789abcdef...

Size: 532 bytes



=== Decoding Attempts ===

RLP decoded: [0x123456... 0xabcdef... ...]
```

3. 데이터베이스 통계

```
$ geth db stats

Compactions

Level | Tables | Size(MB) | Time(sec) | Read(MB) | Write(MB)

-------+------------+---------------+---------------+---------------+-----------

0 | 0 | 0.00000 | 0.00000 | 0.00000 | 0.00000

1 | 8 | 1024.00000 | 45.12300 | 2048.00000 | 1024.00000

2 | 64 | 8192.00000 | 234.56700 | 16384.00000 | 8192.00000
```

4. 메타데이터 조회

```
$ geth db metadata
```

```
=== Chain Metadata ===
Head block: #18500000 (0x1234567890abcdef...)
Genesis block: 0xd4e56740f876aef8c010b86a40d5f56745a118d0906a34e69aec8c0db1cb8
Chain ID: 1
Network ID: 1
Fast sync pivot: #18400000
Snapshot root: 0xabcdef1234567890...
```

🛠️ 문제 해결

일반적인 오류들

1. 데이터베이스 잠금 오류

```
Fatal: Failed to open database: resource temporarily unavailable
```

해결: 다른 geth 프로세스가 실행 중인지 확인, 프로세스 종료 후 재시도

2. 권한 오류

```
Fatal: Failed to open database: permission denied
```

해결: 데이터 디렉토리 권한 확인, 적절한 사용자로 실행

3. 디스크 공간 부족

```
Fatal: Database compaction failed: no space left on device
```

해결: 디스크 공간 확보 후 재시도

🔒 보안 고려사항

위험한 명령어들

- `geth db put` : 데이터베이스 손상 가능
- `geth db delete` : 중요 데이터 삭제 가능
- `geth removedb` : 전체 데이터베이스 삭제

안전한 사용법

1. **백업**: 중요한 작업 전 데이터베이스 백업
2. **테스트**: 테스트넷에서 먼저 시도
3. **확인**: 명령어 실행 전 충분한 검토

참고 자료:

- LevelDB 문서
- Ethereum 데이터베이스 구조
- Geth 데이터베이스 관리

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.